

**UNIVERSITY OF
BUEA**

**FACULTY OF
ENGINEERING AND
TECHNOLOGY
(FET)**



**REPUBLIC OF
CAMEROON**

**PEACE-WORK-
FATHERLAND**

PROJECT TITLE

TASK 4 REPORT

System Modelling and Design

Design and Implementation of a Crowdsensed Drive Test Mobile Application for Coverage Prediction

**COURSE TITLE: INTERNET PROGRAMMING AND
MOBILE PROGRAMMING**

COURSE CODE: CEF 440

COURSE INSTRUCTOR: Dr. Nkameni Valery

ACADEMIC YEAR 2025 - 2026

Project Group: Group 13

Submitted by:

1. Imele Azafa Jose de Kajos **FE23A068**
2. Ayuk Amelia Agbor Oroch **FE23A020**
3. Ngome Fernandez Manyaka **FE23A111**
4. Tadjou Ngnintedem Aubry-Junex **FE23A148**
5. Bessong Takang Tabot Willy **FE23A025**

Abstract

This report presents the system modelling and design phase of CrowdSenseNet, a crowdsensed drive test mobile application built to address the problem of inconsistent and unmapped mobile network coverage in Cameroon. The design follows directly from the requirements specified in Task 3, translating them into a comprehensive set of Unified Modelling Language (UML) diagrams that serve as a blueprint for implementation.

Six diagram types are covered: the Context Diagram, which defines the system boundary and its external actors; the Data Flow Diagram, which shows how information moves through the system; the Use Case Diagram, which maps user interactions to system functions; the Sequence Diagram, which models the time-ordered flow of events for the core data collection scenario; the Class Diagram, which describes the static structure of the system using the MVVM architectural pattern; and the Deployment Diagram, which shows how software components are distributed across physical nodes.

Every design decision in this report is grounded in the stakeholder evidence gathered in Task 2 and the requirements formalised in Task 3. The system is built around three core design priorities: privacy (full anonymisation of all user data), battery efficiency (adaptive sampling below 5% drain per hour), and data quality (GPS accuracy filtering and a minimum 5-second sampling interval).

Contents

Design and Implementation of a Crowdsensed Drive Test Mobile Application	0
for Coverage Prediction	0
Abstract	2
List of Figures	6
List of Tables	6
1.0 Introduction	7
1.1 Project Background	7
1.2 Purpose of This Document	7
1.3 Scope	7
1.4 Definitions and Abbreviations	7
2.0 Modelling Methodology	9
3.0 Context Diagram	10
3.1 What is a Context Diagram?	10
3.2 Why It Matters for CrowdSenseNet	10
3.3 Components of the Context Diagram	10
3.3.1 The System	10
3.3.2 External Entities and Data Flows	10
3.4 Steps Used to Create the Context Diagram	11
3.5 Context Diagram Illustration	12
3.6 Summary	12
4.0 Data Flow Diagram	13
4.1 What is a Data Flow Diagram?	13
4.2 Design Considerations	13
4.3 Level 0 DFD (Context Diagram Reference)	13
4.4 Level 1 DFD: Internal Processes	13
4.4.1 Process 1.0: User Consent and Onboarding	13
4.4.2 Process 2.0: Signal Data Collection	13
4.4.3 Process 3.0: Local Storage Management	14
4.4.4 Process 4.0: Data Upload and Synchronisation	14
4.4.5 Process 5.0: Backend Validation and Storage	14
4.4.6 Process 6.0: Coverage Map Generation	14

4.4.7 Process 7.0: ML Coverage Prediction (Future Phase).....	14
4.5 Key Data Flows	14
5.0 Use Case Diagram	17
5.1 What is a Use Case Diagram?	17
5.2 System Boundary	17
5.3 Actors	17
5.3.1 Primary Actors	17
5.3.2 Secondary Actors	17
5.4 Use Cases	18
5.5 Relationships	19
5.5.1 Include Relationships	19
5.5.2 Extend Relationships	19
5.6 Use Case Diagram Illustration	20
6.0 Sequence Diagram	21
6.1 What is a Sequence Diagram?	21
6.2 Actors and Lifelines	21
6.3 Message Flow	21
6.3.1 Phase 1: App Launch and Consent	21
6.3.2 Phase 2: Data Collection Loop	22
6.3.3 Phase 3: Upload	22
6.3.4 Phase 4: Coverage Map Serving	22
6.4 Design Decisions	23
6.5 Sequence Diagram Illustration	24
7.0 Class Diagram	25
7.1 What is a Class Diagram?	25
7.2 Architectural Overview	25
7.3 Classes by Layer	26
7.3.1 Presentation Layer.....	26
7.3.2 ViewModel Layer	26
7.3.3 Domain / Service Layer	27
7.3.4 Repository Layer	28
7.3.5 Data Layer	28

7.3.6 Network Layer	29
7.3.7 Enumerations.....	30
7.4 Relationships	30
7.4.1 Relationship Types Used.....	30
7.4.2 Dependency Relationships (1 to 1).....	30
7.4.3 Association Relationships	31
7.4.4 Composition and Aggregation Relationships.....	32
7.5 Class Diagram Illustration.....	33
8.0 Deployment Diagram.....	34
8.1 What is a Deployment Diagram?	34
8.2 Architecture Overview	34
8.3 Node Breakdown	34
8.3.1 Android Device	34
8.3.2 Android System Services	35
8.3.3 Backend Server	35
8.3.4 ML Analytics Server (Future Phase).....	35
8.4 Inter-Node Communication Paths	36
8.5 Modelling Tool and Challenges.....	36
8.6 Deployment Diagram Illustration	37
9.0 Member Contributions	38
10.0 Conclusion	40

List of Figures

Figure 1: Steps to create a context diagram	
Figure 2: Context Diagram for CrowdSenseNet.....	
Figure 3: Level 1 Data Flow Diagram for CrowdSenseNet	
Figure 4: Use Case Diagram for CrowdSenseNet	
Figure 5: Sequence Diagram: Core Data Collection Scenario.....	
Figure 6: Class Diagram for CrowdSenseNet (MVVM)	
Figure 7: Deployment Diagram for CrowdSenseNet.....	

List of Tables

Table 1: UML diagram types and their purpose in this project	
Table 2: External entities and their data flows.....	
Table 3: Key data flows in the Level 1 DFD	
Table 4: Use cases and their descriptions	
Table 5: Actor justification	
Table 6: Sequence diagram design decisions.....	
Table 7: CrowdSenseNet class summary by layer.....	
Table 8: Dependency relationships	
Table 9: Association relationships	
Table 10: Composition and aggregation relationships.....	
Table 11: Deployment nodes and their roles	
Table 12: Inter-node communication paths	
Table 13: Member contribution summary	

1.0 Introduction

1.1 Project Background

Mobile network users in Cameroon face daily problems: weak signals, sudden call drops, slow internet speeds, and coverage gaps in both rural and urban areas. At the same time, the traditional way of measuring coverage, sending engineers out in a vehicle with specialised equipment (called drive testing), is expensive, slow, and only covers the roads the engineers travel on. It cannot capture indoor coverage, pedestrian areas, or the experience of ordinary people going about their day.

CrowdSenseNet addresses this problem by turning ordinary Android smartphones into passive signal measurement devices. Every user who installs the app automatically contributes real signal readings from their location, building a continuously updated, crowd-sourced coverage map. This approach is cheaper, broader, and more representative than traditional drive testing.

1.2 Purpose of This Document

This document presents the complete system modelling and design for CrowdSenseNet. It translates the requirements formalised in Task 3 into a set of UML diagrams that serve as the architectural blueprint for implementation. Each diagram addresses a different aspect of the system: its boundaries, its data flows, its user interactions, its runtime behaviour, its static structure, and its physical deployment.

1.3 Scope

The design covers the full CrowdSenseNet system as defined by the Task 3 SRS, including the Android mobile application, the Python FastAPI backend server, the PostgreSQL/PostGIS database, and the machine learning analytics layer (modelled but deferred to a future phase). iOS support is excluded from this phase. The geographic scope is limited to Cameroon.

1.4 Definitions and Abbreviations

Term / Abbreviation	Definition
RSRP	Reference Signal Received Power. Primary LTE signal strength metric. Coverage hole threshold: below -110 dBm
RSRQ	Reference Signal Received Quality. Measures signal quality relative to interference
SINR	Signal to Interference-plus-Noise Ratio. Indicates signal clarity
UML	Unified Modelling Language. Standardised notation for system design
MVVM	Model-View-ViewModel. Architectural pattern used in the Android app

Term / Abbreviation	Definition
DFD	Data Flow Diagram. Shows how data moves through a system
UUID	Universally Unique Identifier. Random anonymous token generated on first launch
SRS	Software Requirements Specification. Produced in Task 3
GPS	Global Positioning System. Used for location tagging of signal readings
API	Application Programming Interface. Used to access hardware and backend services
KPI	Key Performance Indicator. Signal metrics like RSRP used by network engineers
MVP	Minimum Viable Product. The core set of must-have features for the first release
ML	Machine Learning. Used for coverage hole prediction
GeoJSON	Open standard format for geographic data, preferred by network engineers

Table 1.1: Key definitions and abbreviations

2.0 Modelling Methodology

The modelling and design phase translates the requirements gathered in Task 2 and formalised in Task 3 into a structured visual blueprint. We used the Unified Modelling Language (UML), a standardised international framework for representing system architecture, behaviour, and structure.

The process was iterative. The team started with the highest-level view of the system (the context diagram) and progressively moved to more detailed representations (class and deployment diagrams). This top-down approach ensures that every detail in the lower-level diagrams can be traced back to the system boundary and stakeholder requirements established at the top.

Diagram	Type	Purpose in This Project
Context Diagram	Structural	Defines the system boundary and shows all external actors and their data flows with CrowdSenseNet
Data Flow Diagram	Structural	Shows how data moves through internal processes, data stores, and external entities
Use Case Diagram	Behavioural	Captures the functional interactions between actors and the system
Sequence Diagram	Behavioural	Models the time-ordered message flow for the core data collection scenario
Class Diagram	Structural	Defines the static object-oriented structure of the Android app following the MVVM pattern
Deployment Diagram	Structural	Maps software components onto physical execution environments across the three-tier architecture

Table 2.1: UML diagram types and their purpose in this project

Diagrams were created using draw.io and EdrawMax. All design decisions are grounded in evidence from Task 2 stakeholder data and are traceable to specific requirements in the Task 3 SRS.

3.0 Context Diagram

Prepared by: Tadjou Ngintedem Aubry-Junex

3.1 What is a Context Diagram?

A context diagram is the highest-level view of any system. It shows the entire system as a single process at the centre, surrounded by all the external entities that send data to it or receive data from it. The diagram does not show any internal processes or details; its only job is to make the system boundary clear.

In system modelling, the context diagram is always the starting point. It ensures that everyone involved, whether developers, engineers, regulators, or supervisors, agrees on what belongs inside the system and what sits outside it before any detailed design work begins.

3.2 Why It Matters for CrowdSenseNet

CrowdSenseNet involves four very different groups of people: ordinary smartphone users who contribute data, network engineers who consume the coverage maps, government regulators who enforce privacy rules, and academic supervisors who evaluate the project. Without a context diagram, it is easy for different people to have different ideas about what the system is supposed to do and who it talks to. The context diagram gives everyone a single shared picture before any detailed design begins.

It also makes sure that every requirement from the Task 3 SRS is linked to a real external actor, preventing scope creep and ensuring no stakeholder has been forgotten.

3.3 Components of the Context Diagram

3.3.1 The System

At the centre is the CrowdSenseNet System, which includes the Android mobile application and the backend server. The system collects signal data, stores it locally, uploads it to the backend, generates coverage maps, and provides reports.

3.3.2 External Entities and Data Flows

External Entity	Data Sent to System	Data Received from System
Users (Smartphone Owners)	Signal readings (RSRP, RSRQ, SINR, GPS), consent confirmation, on/off toggle commands	Coverage feedback display, survey prompts, upload status notifications

External Entity	Data Sent to System	Data Received from System
Network Engineers (MTN, Orange)	Coverage map requests, export requests	Processed coverage heatmaps, GeoJSON/CSV data exports
Regulators (Telecom Authorities)	Privacy guidelines and compliance requirements	Anonymisation compliance reports
Supervisors (Lecturers)	Validation feedback and academic review comments	Project documentation, methodology reports

Table 3.1: External entities and their data flows

3.4 Steps Used to Create the Context Diagram

1. Define the system boundary: identify what belongs inside CrowdSenseNet (the mobile app and backend) and what stays outside.
2. Identify all external actors: list every person or organisation that sends data to or receives data from the system.
3. Determine inputs and outputs: for each actor, specify exactly what data flows in and what flows out.
4. Draw the data flow arrows: show the direction of communication between the system and each actor.
5. Validate with stakeholders: confirm the diagram is complete and accurate before proceeding to the DFD.

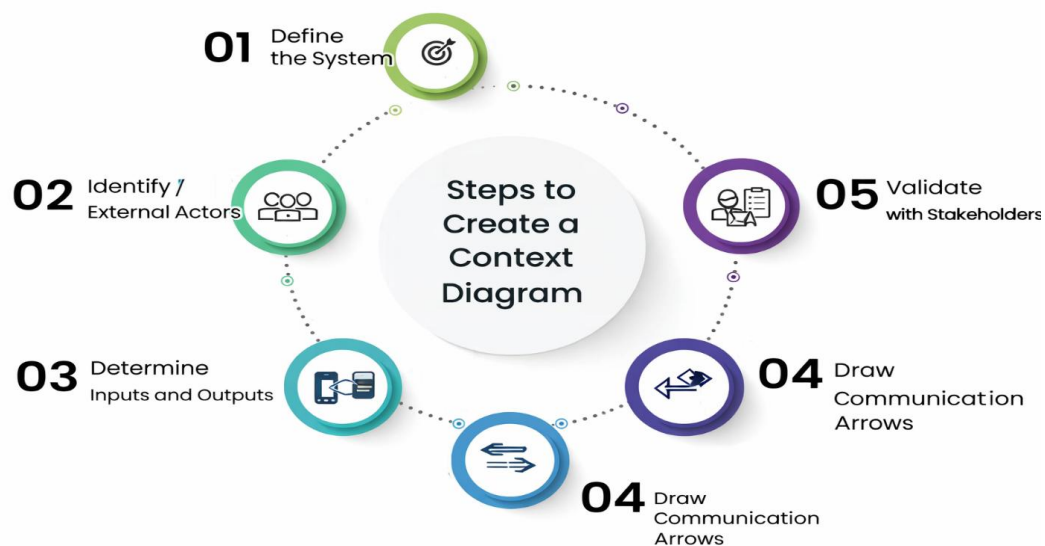


Figure 1: Steps to create a context diagram

3.5 Context Diagram Illustration

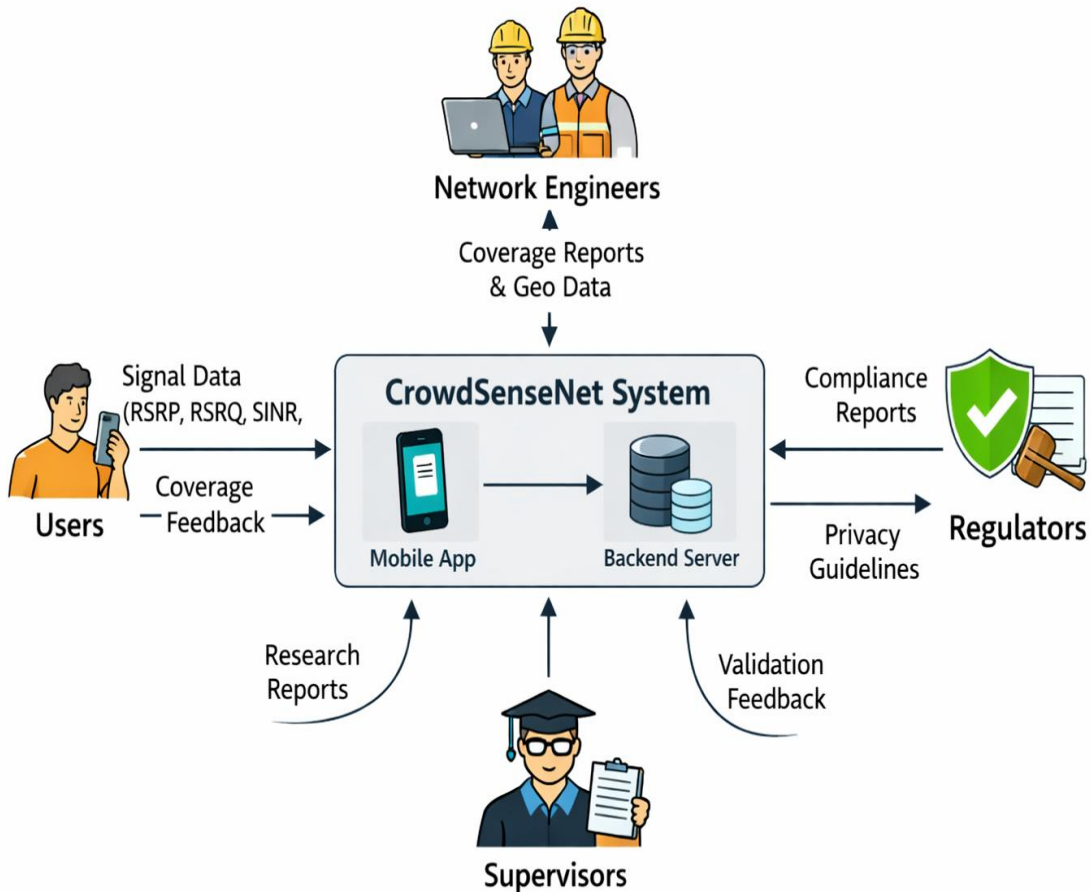


Figure 2: Context Diagram for CrowdSenseNet

3.6 Summary

The context diagram establishes the boundary of CrowdSenseNet and confirms that all four stakeholder groups from the Task 3 SRS are accounted for. Users feed raw signal data in and receive coverage feedback out. Engineers receive processed maps. Regulators receive compliance assurance. Supervisors receive documentation. This shared understanding is the foundation for every diagram that follows.

4.0 Data Flow Diagram

Prepared by: Imele Azafa Jose de Kajos

4.1 What is a Data Flow Diagram?

A Data Flow Diagram (DFD) shows how data moves through a system. While the context diagram (Level 0) shows the system as a single black box, the Level 1 DFD opens that box and shows the internal processes, the data stores, and how data flows between them and the external actors.

The DFD uses four elements: external entities (people or systems outside the boundary), processes (transformations that act on data), data stores (where data is held), and data flows (arrows showing the movement of data).

4.2 Design Considerations

- Clarity: every process, data store, and data flow is clearly labelled with a plain English name.
- Completeness: all data flows visible in the context diagram must appear at Level 1.
- No control flow: the DFD shows what data moves, not when or why. Decision logic belongs in sequence diagrams.
- Numbered processes: each process is numbered to make traceability easier.
- Consistent notation: standard DFD symbols are used throughout (rounded rectangles for processes, open-ended rectangles for data stores, squares for external entities).

4.3 Level 0 DFD (Context Diagram Reference)

The Level 0 DFD is the context diagram already presented in Section 3. It shows CrowdSenseNet as a single process receiving signal data from users and returning coverage information. The Level 1 DFD below decomposes that single process into its internal parts.

4.4 Level 1 DFD: Internal Processes

4.4.1 Process 1.0: User Consent and Onboarding

On first launch, the app presents a consent screen to the user. The user grants location and phone state permissions. The app generates a random anonymous UUID and stores it in local encrypted preferences. No personal information is collected at this stage. Output: a confirmed consent record and a UUID stored in the local preferences data store.

4.4.2 Process 2.0: Signal Data Collection

This is the core process of the system. When the user starts a session, the app calls the Android TelephonyManager API to read RSRP, RSRQ, SINR, and RSSI, and the FusedLocationProvider API to get GPS coordinates and accuracy. If the device is moving (speed above 5 km/h), readings are taken every 10 seconds. If the device is stationary for more than 30 seconds, collection pauses automatically. Any

GPS reading with an accuracy worse than 15 metres is discarded. Valid readings are written immediately to the local Room database.

4.4.3 Process 3.0: Local Storage Management

All signal readings are stored in the local Room database before any upload is attempted. This ensures offline operation. The local database is capped at 50MB. When the cap is reached, the oldest readings that have already been uploaded are deleted automatically. Unsynced readings are never deleted.

4.4.4 Process 4.0: Data Upload and Synchronisation

The SyncManager monitors WiFi connectivity. When WiFi is available (default) or when the user has enabled mobile data upload, the app sends a batch of unsynced readings to the backend server over HTTPS/TLS 1.2. If the upload fails, the app retries automatically after 30 minutes. After a successful upload, the readings are marked as synced in the local database.

4.4.5 Process 5.0: Backend Validation and Storage

The backend FastAPI server receives the uploaded reading batch. It validates each reading: it checks that the GPS coordinates fall within the Cameroon bounding box, averages readings taken within a 10-metre radius of each other to remove duplicates, and verifies the authentication token. Valid readings are written to the PostgreSQL/PostGIS database.

4.4.6 Process 6.0: Coverage Map Generation

The backend generates a coverage heatmap by querying the PostGIS spatial database for readings within the requested area. Signal values are classified into good, average, poor, and coverage hole zones (RSRP below -110 dBm). The output is a GeoJSON feature collection that the app displays on the Map screen and that engineers can download as GeoJSON or CSV.

4.4.7 Process 7.0: ML Coverage Prediction (Future Phase)

Once sufficient data has accumulated (at least 50 readings per 1 km squared), the ML analytics server will classify unmeasured areas based on interpolated signal patterns. This process is modelled in the deployment diagram but is deferred to a future development phase as a Won't Have (now) requirement from the Task 3 MoSCoW prioritisation.

4.5 Key Data Flows

Data Flow	Source	Destination	Description
Consent confirmation	User	Process 1.0	User grants permissions on first launch

Data Flow	Source	Destination	Description
UUID	Process 1.0	Local Prefs store	Anonymous device token generated once and stored
TelephonyManager read	Process 2.0	Android hardware	App requests RSRP, RSRQ, SINR, RSSI from cellular modem
Signal metrics	Android hardware	Process 2.0	Raw KPI values returned by the TelephonyManager API
GPS fix	Android hardware	Process 2.0	Latitude, longitude, and accuracy estimate from FusedLocationProvider
Filtered signal reading	Process 2.0	Local DB	Validated reading (GPS accuracy passed) written to Room database
Unsynced readings	Local DB	Process 4.0	Batch of readings not yet uploaded to the backend
Upload batch (HTTPS)	Process 4.0	Backend server	Encrypted batch of anonymised readings sent over TLS 1.2
Upload acknowledgement	Backend server	Process 4.0	Server confirms successful receipt; app marks readings as synced
Validated readings	Process 5.0	PostgreSQL DB	Geographically valid, deduplicated readings stored server-side
Coverage map query	Engineer / App	Process 6.0	Request for heatmap data for a specific area
GeoJSON heatmap	Process 6.0	App / Engineer	Coverage map data returned for display or download
Session on/off toggle	User	Process 2.0	User starts or stops data collection from the Home screen

Data Flow	Source	Destination	Description
Upload status notification	Process 4.0	User	App notifies user when data has been uploaded or queued

Table 4.1: Key data flows in the Level 1 DFD

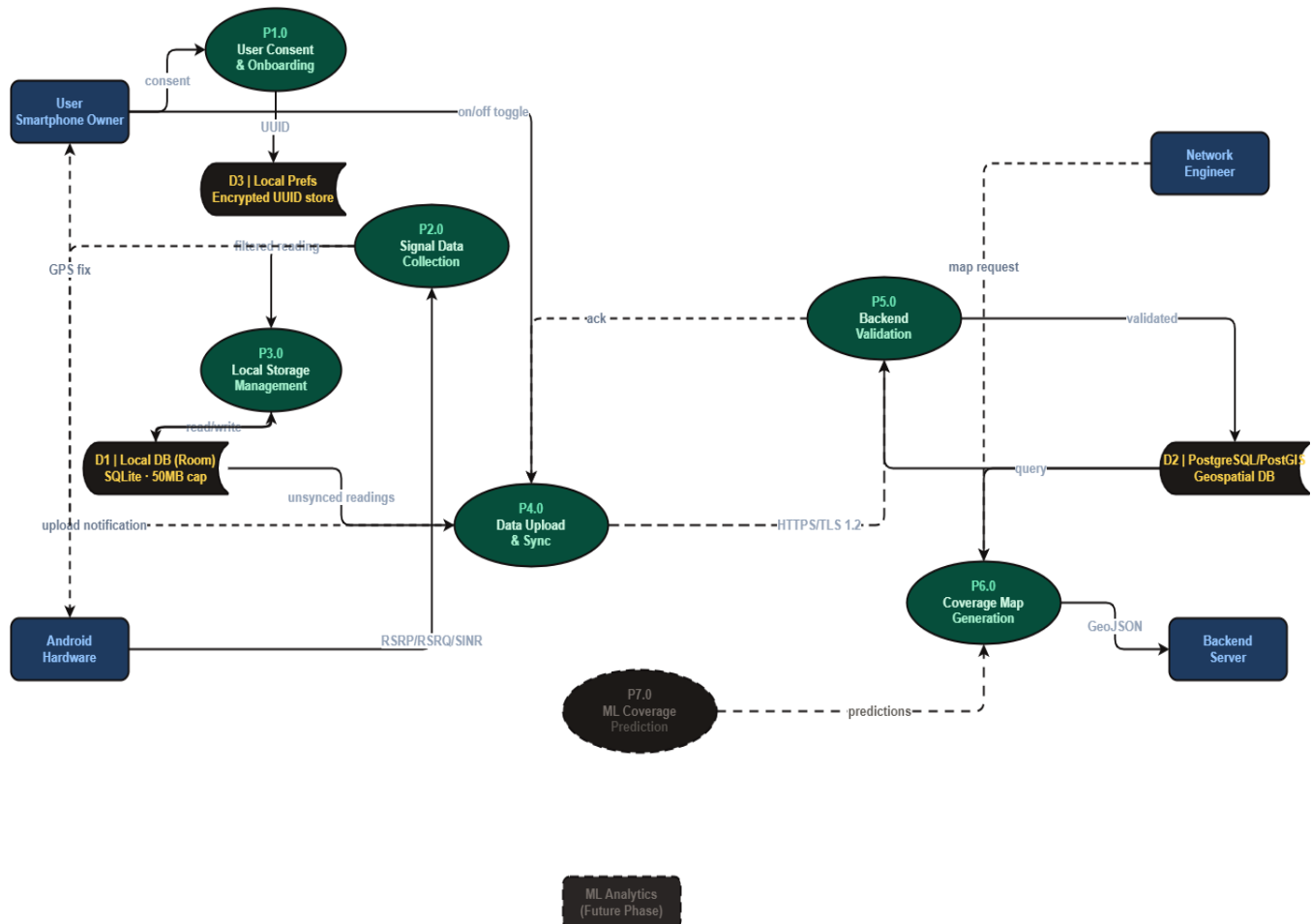


Figure 3: Level 1 Data Flow Diagram for CrowdSenseNet

5.0 Use Case Diagram

Prepared by: Imele Azafa Jose de Kajos

5.1 What is a Use Case Diagram?

A use case diagram is a behavioural UML diagram that shows what a system does from the perspective of its users. It identifies the actors (the people or systems that interact with the application) and the use cases (the actions or functions they perform). It does not show how those functions are implemented internally; that is the job of the sequence and class diagrams.

5.2 System Boundary

The system boundary defines what is inside CrowdSenseNet and what is outside. Everything inside the boundary is a function the app provides. Everything outside is an actor that uses those functions. The system boundary for this project encloses the CrowdSenseNet mobile application and backend.

5.3 Actors

5.3.1 Primary Actors

Actor	Role	Justification for Inclusion
User (Smartphone Owner)	Primary data contributor. Uses the app to contribute signal data passively.	The app cannot function without users contributing data. All core functional requirements (FR-01 to FR-12) are driven by user interaction.
Network Engineer (MTN / Orange)	Primary data consumer. Uses coverage maps and exported data for network planning.	Engineers were the primary source of technical requirements (KPIs, GPS threshold, sampling interval). They are the main professional beneficiaries of the system.
Android System Services	Non-human actor. Provides hardware access (TelephonyManager, FusedLocationProvider).	The app depends on these OS-level APIs for all signal and location data. They are an essential technical actor even though they are not human.

Table 5.1: Primary actors and their justification

5.3.2 Secondary Actors

- Regulators: provide privacy guidelines that shape how the system handles data. They do not interact with the app directly but constrain its design.
- Supervisors: review documentation and validate the academic methodology. They interact with project outputs rather than the live system.

5.4 Use Cases

Use Case	Actor(s)	Description
Start / Stop Data Collection	User	User taps a single button to begin or end a data collection session (FR-01)
View Live Signal Dashboard	User	User sees real-time RSRP, RSRQ, SINR, and GPS status on the Home screen
View Coverage Heatmap	User, Network Engineer	Both actors can view the colour-coded coverage map showing good, average, poor, and coverage hole zones
View Session History	User	User browses past sessions with date, duration, reading count, and upload status
Configure Settings	User	User adjusts collection mode (background/foreground), sampling interval, upload mode (WiFi only or mobile data), and language
Upload Data to Server	User (triggered), Android System Services (automated)	App automatically uploads buffered readings to the backend when connectivity conditions are met
Export Coverage Data	Network Engineer	Engineer downloads coverage data in GeoJSON or CSV format from the web interface (FR-12)
Grant Permissions	User	On first launch, user grants location and phone state permissions through the onboarding screen (FR-08)
Collect Signal Reading	Android System Services	TelephonyManager reads RSRP, RSRQ, SINR, and RSSI every 10 seconds while the device is moving (FR-04)

Use Case	Actor(s)	Description
Provide GPS Fix	Android System Services	FusedLocationProvider returns latitude, longitude, and accuracy for each reading (FR-05)
Anonymise and Store Reading	User (implicit)	App strips all PII and writes the anonymised reading with a UUID token to the local database (FR-07, FR-08)
Sync Data on Retry	Android System Services	WorkManager retries failed uploads every 30 minutes automatically (FR-11)

Table 5.2: Use cases, actors, and descriptions

5.5 Relationships

5.5.1 Include Relationships

- Start Data Collection includes Collect Signal Reading (data collection always triggers a reading cycle)
- Collect Signal Reading includes Provide GPS Fix (every reading requires a GPS tag)
- Collect Signal Reading includes Anonymise and Store Reading (every reading is anonymised before storage)
- Upload Data to Server includes Sync Data on Retry (retry logic is always part of the upload process)

5.5.2 Extend Relationships

- View Coverage Heatmap may extend Export Coverage Data (engineers can optionally export what they are viewing)
- Configure Settings may extend Enable Mobile Data Upload (mobile data upload is an optional extension of the default WiFi-only setting)

5.6 Use Case Diagram Illustration

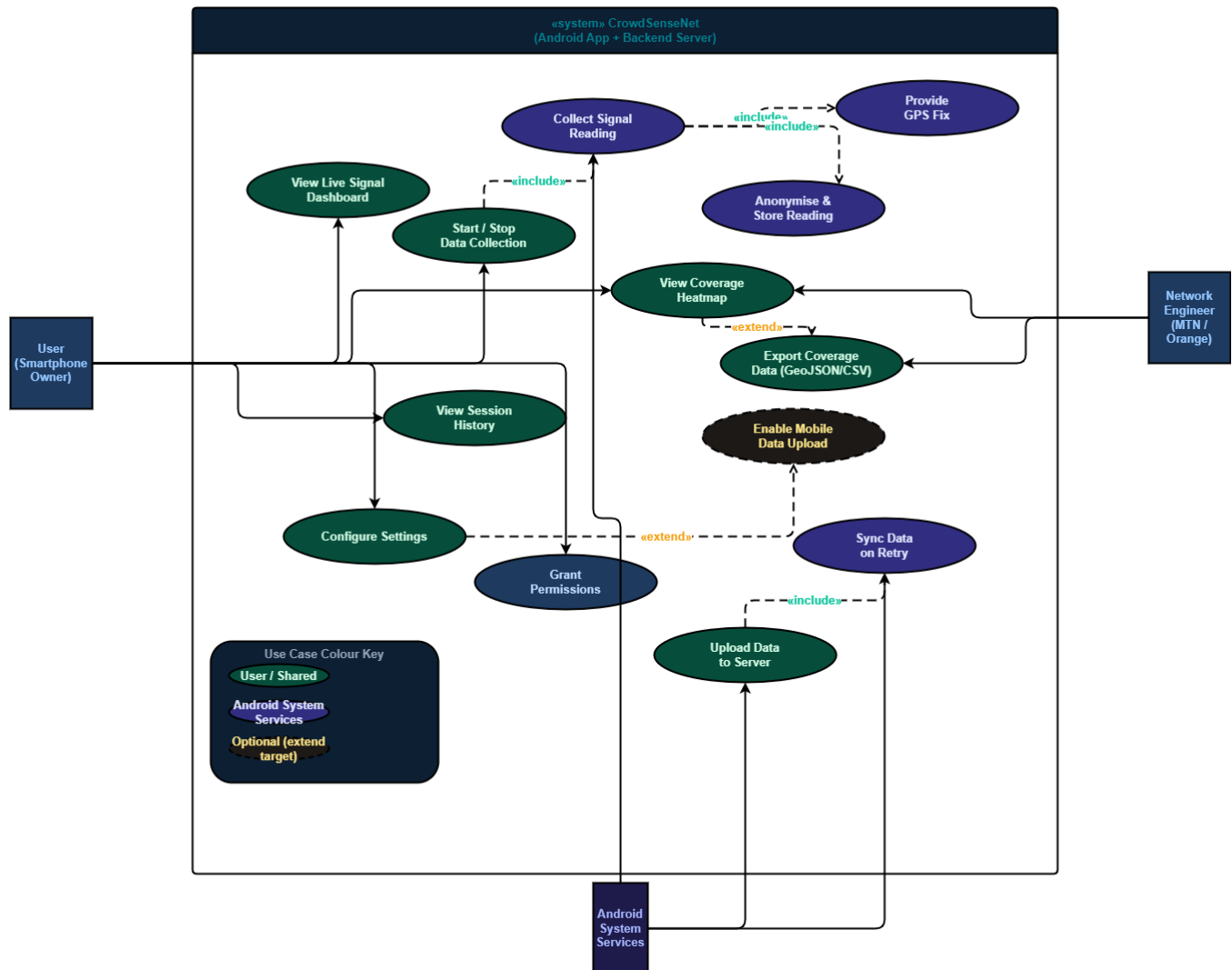


Figure 4: Use Case Diagram for CrowdSenseNet

6.0 Sequence Diagram

Prepared by: Bessong Takang Tabot Willy

6.1 What is a Sequence Diagram?

A sequence diagram is a behavioural UML diagram that models the time-ordered flow of messages between the actors and components of a system for a specific scenario. Unlike the use case diagram, which shows what the system does, the sequence diagram shows exactly how it happens, step by step, in the right order.

6.2 Actors and Lifelines

Six lifelines are defined for the core data collection scenario. Each represents a distinct actor or system component that participates in the interaction.

Lifeline	Type	Role in the Scenario
User	Human actor	The smartphone owner who opens the app, grants consent, starts and stops collection, and receives feedback
Mobile App	System component	The CrowdSenseNet Android application. Acts as the central coordinator throughout the session
Telephony API	System component	The Android TelephonyManager. Supplies RSRP, SINR, RSRQ, and RSSI values on request
GPS Module	System component	The Android FusedLocationProvider. Returns latitude, longitude, and fix accuracy
Backend Server	System component	The FastAPI cloud REST API. Receives, validates, and stores uploaded measurements
Database	System component	The PostgreSQL/PostGIS store on the backend. Persists all readings and serves heatmap queries

Table 6.1: Lifelines in the sequence diagram

6.3 Message Flow

6.3.1 Phase 1: App Launch and Consent

The user opens the application. The Mobile App displays a consent screen and requests location and phone state permissions from Android. Once the user grants permissions, the app confirms consent and activates the background sensor loop.

6.3.2 Phase 2: Data Collection Loop

This phase is modelled as a loop combined fragment that repeats every 10 seconds while the device is moving.

6. The Mobile App sends a read request to the Telephony API.
7. The Telephony API returns RSRP, SINR, and RSRQ values.
8. The Mobile App simultaneously requests a GPS fix from the GPS Module.
9. The GPS Module returns latitude, longitude, and an accuracy estimate.
10. The Mobile App evaluates the accuracy value. Any fix worse than 15 metres is discarded silently without interrupting the loop.
11. Valid readings are anonymised on-device: all device-identifiable fields are removed and the session UUID is assigned.
12. The anonymised GeoJSON record is written to the local Room database.
13. The Home screen updates to show the latest signal values to the user.

At any point during the loop, the user may send a toggle command to pause or resume collection. The Mobile App updates its state immediately.

If the device is stationary for more than 30 seconds, the app pauses the collection loop automatically and resumes when movement is detected (speed above 5 km/h). This is the primary mechanism for meeting the battery efficiency requirement.

6.3.3 Phase 3: Upload

The upload phase is modelled as an alt combined fragment with two branches.

WiFi available branch: the WorkManager service detects a WiFi connection and dispatches the buffered GeoJSON batch to the Backend Server over HTTPS/TLS 1.2. The backend validates and cleans the data, writes each record to the Database, receives an OK confirmation, and returns an upload acknowledgement to the Mobile App. The app notifies the user that the upload was successful and marks the readings as synced in the local database.

No WiFi / WiFi-only restriction active branch: the app queues the data locally and notifies the user that the upload has been deferred. The WorkManager schedules a retry in 30 minutes.

6.3.4 Phase 4: Coverage Map Serving

When the Map screen is opened, the Mobile App queries the Backend Server for the latest coverage data. The Backend Server requests the relevant coverage data from the Database, which returns a GeoJSON export. The backend renders the heatmap and returns it to the Mobile App for display. When a network engineer queries the system through the web interface, the same flow applies.

6.4 Design Decisions

Decision	Rationale / Evidence
10-second collection interval while moving, pause when stationary	Balances the conflict identified in Task 3 between real-time data collection and battery efficiency. Engineers confirmed 5 seconds as the minimum; 10 seconds during movement with pausing when stationary keeps battery drain below 5% per hour.
GPS accuracy filtering at 15 metres, applied before storage	Threshold set by network engineer interviews. Applied on-device before the record is written to avoid corrupting the coverage map with inaccurate coordinates.
WiFi-only upload as default, mobile data as opt-in toggle	56% of survey users required data-saving conditions. Modelled as an alt fragment to make both execution paths explicitly testable.
Data anonymisation performed on-device before any record leaves local storage	Non-negotiable requirement for 56 of 100 survey respondents. Removing PII on the device means the backend never receives identifiable data, not even in transit.
Activation bars on Mobile App for full session duration	Reflects the app's role as the persistent central coordinator. Backend and database lifelines activate only during upload and query phases, correctly showing their idle periods.
Random UUID generated on first launch, stored in encrypted SharedPreferences	Resolves the conflict between full anonymity and device-consistent readings for the ML model (Task 3, Section 2.1.3).

Table 6.2: Sequence diagram design decisions and evidence

6.5 Sequence Diagram Illustration

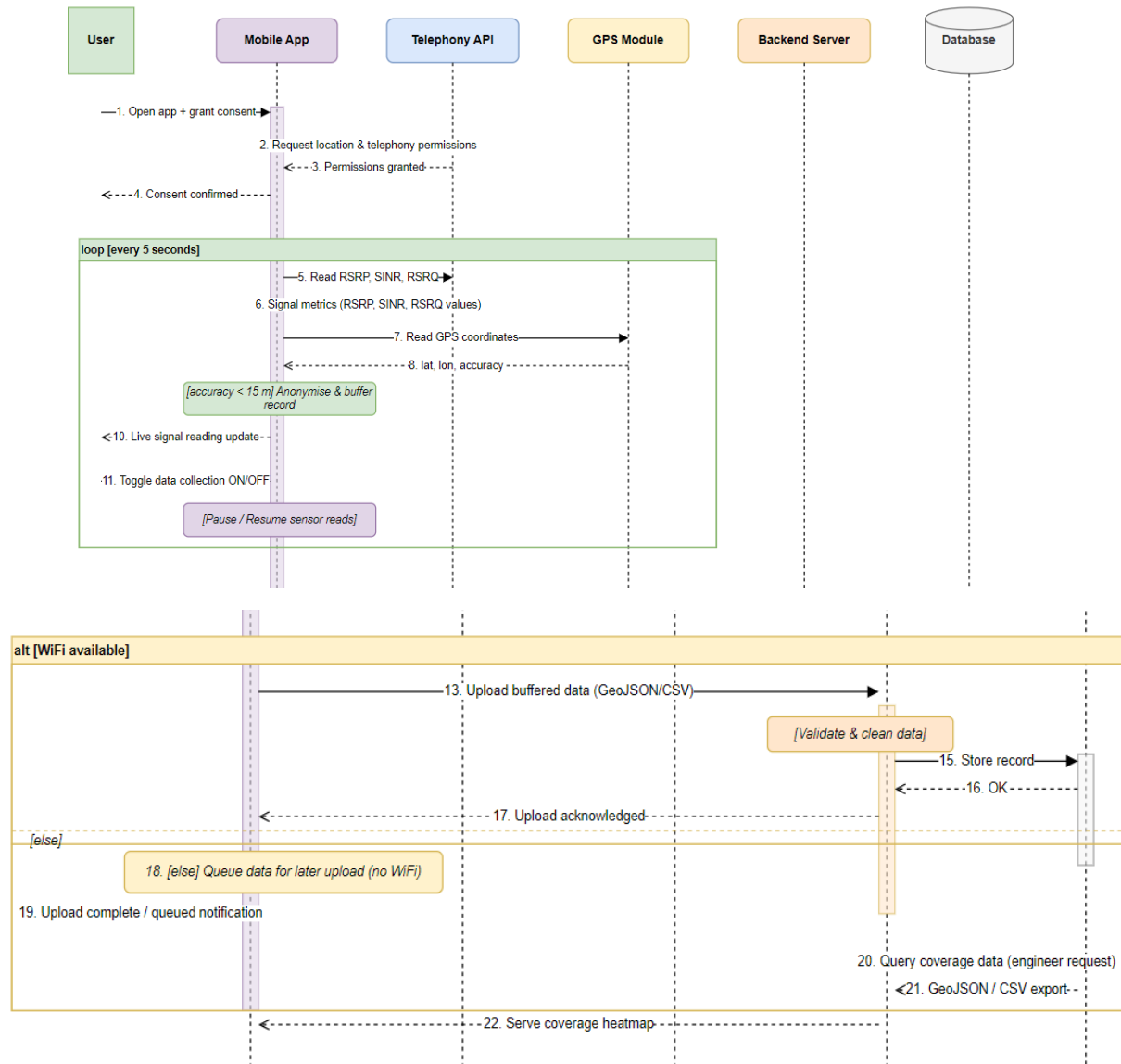


Figure 5: Sequence Diagram for the Core Data Collection Scenario

7.0 Class Diagram

Prepared by: Ayuk Amelia Agbor Orock

7.1 What is a Class Diagram?

A class diagram is a structural UML diagram that describes the static structure of a system. It shows the classes (the building blocks of the code), the attributes each class holds, the operations each class can perform, and the relationships between classes. It is produced before any code is written so that the development team has a shared blueprint for the architecture.

7.2 Architectural Overview

CrowdSenseNet follows the MVVM (Model-View-ViewModel) architectural pattern, which is the standard recommended architecture for Android applications built with Kotlin. MVVM separates the application into three concerns: what the user sees (the View), the data and business logic (the Model), and the layer that connects them (the ViewModel). This separation makes the codebase easier to test, maintain, and extend.

The full system is divided into seven packages, each with a single clear responsibility.

Package	Responsibility	Contents
Presentation Layer	Screens the user sees and interacts with	5 screen classes
ViewModel Layer	Holds live data and connects screens to business logic	4 ViewModel classes
Domain / Service Layer	Core workers: signal reading, syncing, permissions	4 classes + 2 interfaces
Repository Layer	Bridge between ViewModels and data sources	3 Repository classes
Data Layer	Data objects stored in the local database	5 entity classes
Network Layer	All communication with the backend server	3 network classes
Enumerations	Fixed value sets used across the system	3 enumerations

Table 7.1: CrowdSenseNet package structure

7.3 Classes by Layer

7.3.1 Presentation Layer

These classes handle only what the user sees. No business logic or database calls appear here. Each screen class observes its corresponding ViewModel and updates automatically when data changes.

Class	Purpose
OnboardingScreen	Shown once on first launch. Walks the user through what the app does, what data it collects, and requests the required Android permissions.
HomeScreen	The main screen. Shows live signal metrics (RSRP, RSRQ, SINR, RSSI), GPS status, current session status, and a start/stop button.
MapScreen	Displays the real-time coverage heatmap. Colour-coded zones show good, average, poor, and insufficient data areas. Users can tap any zone to see details.
HistoryScreen	Shows a list of all past data collection sessions with their date, duration, number of readings, and upload status.
SettingsScreen	Allows the user to configure collection mode, enable or disable mobile data upload, change language, and delete all locally stored data.

Table 7.2: Presentation layer classes

7.3.2 ViewModel Layer

ViewModels sit between the screens and the data. They hold LiveData objects that the screens observe and automatically update when values change. The screen never fetches data directly; it watches the ViewModel and reacts to changes.

Class	Purpose
HomeViewModel	Provides the HomeScreen with live values for RSRP, RSRQ, SINR, RSSI, session status, and total readings.

Class	Purpose
MapViewModel	Fetches ML predictions from the PredictionRepository and provides them to the MapScreen for heatmap display.
HistoryViewModel	Retrieves all past sessions from the SessionRepository and provides them as a live list.
SettingsViewModel	Loads and saves user preferences through the User entity. Handles the delete-all-data action.

Table 7.3: ViewModel layer classes

7.3.3 Domain / Service Layer

This layer contains the core workers of the application. These classes run the actual business logic: reading signal data, managing GPS, checking permissions, and uploading data.

Class / Interface	Type	Purpose
DataCollector	Interface	Defines the contract for any data-collecting class. Any implementation must provide startCollection(), stopCollection(), pauseCollection(), and resumeCollection() methods.
DataUploader	Interface	Defines the contract for any data-uploading class. Requires uploadData(), checkConnectivity(), and scheduleRetry() methods.
SignalCollector	Class	Implements DataCollector. Reads RSRP, RSRQ, SINR, RSSI, and Cell ID via TelephonyManager every 10 seconds while moving. Creates SignalReading objects and saves them to the local database.
SyncManager	Class	Implements DataUploader. Monitors WiFi connectivity and uploads queued readings to the backend. Retries automatically every 30 minutes if upload fails.
PermissionManager	Class	Handles Android permission requests for location access, phone state, and battery optimisation exemption.

Class / Interface	Type	Purpose
LocationManager	Class	Manages GPS updates via FusedLocationProvider. Detects GPS disabled state and notifies the app to pause collection.

Table 7.4: Domain / Service layer classes and interfaces

7.3.4 Repository Layer

Repository classes act as the single source of truth for data. The ViewModel asks the Repository for data, and the Repository decides whether to retrieve it from the local database or the backend. The ViewModel never knows where the data comes from.

Class	Purpose
SignalRepository	Saves signal readings to the local database, retrieves unsynced readings for upload, and marks readings as synced after successful upload.
SessionRepository	Creates new sessions, ends sessions, retrieves all past sessions, and deletes sessions from the local database.
PredictionRepository	Fetches ML coverage predictions from the backend through the BackendAPI and provides them to the MapViewModel.

Table 7.5: Repository layer classes

7.3.5 Data Layer

These are the data entities, the actual objects stored in the local Room database and passed between layers. Each entity maps directly to a table in the SQLite database.

Class	Key Attributes	Purpose
SignalReading	id, deviceUUID, rsrp, rsrq, sinr, rssi, cellId, latitude, longitude, timestamp, networkType, isSynced, sessionId	One complete signal measurement. Every reading includes both signal parameters and GPS coordinates.

Class	Key Attributes	Purpose
Session	sessionId, deviceUUID, startTime, endTime, totalReadings, uploadStatus, isActive	One data collection session from start tap to stop tap.
User	deviceUUID, language, collectionMode, wifiOnlyUpload, intervalSeconds, hasCompletedOnboarding	Stores the anonymous device token and all user preferences. No personal information.
MLPrediction	predictionId, latitude, longitude, predictedRsrp, coverageClass, confidenceScore, predictionDate	One coverage prediction for a specific geographic location.
LocalDatabase	dbInstance, storageUsedMB, maxStorageMB	The Room database instance. Manages all local storage with a 50MB cap.

Table 7.6: Data layer entity classes

7.3.6 Network Layer

This layer handles all communication between the Android app and the remote backend. All transmissions use HTTPS/TLS 1.2.

Class	Purpose
BackendAPI	Sends uploaded signal readings to the backend and fetches ML predictions. Handles authentication tokens and error responses.
APIResponse	Represents the server response after each API call. Contains the status code, success flag, message, and any returned data.
APIError	Represents an error from the backend. Distinguishes network errors from server errors and determines whether the call should be retried.

Table 7.7: Network layer classes

7.3.7 Enumerations

Enumeration	Values	Used In
CoverageClass	GOOD, AVERAGE, POOR, HOLE, INSUFFICIENT_DATA	MLPrediction: classifies predicted signal quality
NetworkType	LTE, NR_NSA, NR_SA, UMTS, EDGE, UNKNOWN	SignalReading: records the network generation at the time of measurement
CollectionMode	BACKGROUND, FOREGROUND_ONLY, PAUSED	User: stores whether the app collects data always, only when open, or not at all

Table 7.8: Enumerations used in the class diagram

7.4 Relationships

7.4.1 Relationship Types Used

The class diagram uses five relationship types: Dependency (dashed arrow, one class temporarily uses another), Association (solid arrow, one class holds a reference to another), Aggregation (hollow diamond, one class contains another but the child can exist independently), Composition (filled diamond, the child cannot exist without the parent), and Realisation (dashed arrow with hollow head, a class implements an interface).

7.4.2 Dependency Relationships (1 to 1)

From	To	Label
HomeScreen	HomeViewModel	observes
MapScreen	MapViewModel	observes
HistoryScreen	HistoryViewModel	observes
SettingsScreen	SettingsViewModel	observes
OnboardingScreen	PermissionManager	requests

From	To	Label
HomeScreen	SignalCollector	controls
SignalCollector	PermissionManager	requires
SyncManager	PermissionManager	checks
BackendAPI	APIResponse	returns
BackendAPI	APIError	handles
User	CollectionMode	uses enum
MLPrediction	CoverageClass	uses enum
SignalReading	NetworkType	uses enum

Table 7.9: Dependency relationships

7.4.3 Association Relationships

From	To	Cardinality	Label
HomeViewModel	SignalRepository	1 to 1	uses
HomeViewModel	SessionRepository	1 to 1	uses
MapViewModel	PredictionRepository	1 to 1	uses
HistoryViewModel	SessionRepository	1 to 1	uses
SettingsViewModel	User	1 to 1	manages
SignalRepository	LocalDatabase	1 to 1	queries
SessionRepository	LocalDatabase	1 to 1	queries
PredictionRepository	BackendAPI	1 to 1	fetches from

From	To	Cardinality	Label
SignalCollector	LocationManager	1 to 1	uses
SyncManager	SignalRepository	1 to 1	reads from
SyncManager	BackendAPI	1 to 1	calls
User	Session	1 to 0..*	starts
BackendAPI	MLPrediction	1 to 1..*	triggers

Table 7.10: Association relationships

7.4.4 Composition and Aggregation Relationships

Type	From	To	Cardinality	Meaning
Composition	Session	SignalReading	1 to 1..*	A session owns its readings. If a session is deleted, all its readings are deleted too.
Composition	SignalCollector	SignalReading	1 to 1..*	The signal collector creates and owns readings during active collection.
Aggregation	LocalDatabase	SignalReading	1 to 0..*	The database stores readings but a reading can exist in memory before it is committed.
Aggregation	LocalDatabase	Session	1 to 0..*	The database stores sessions but a session object exists in memory before storage.

Table 7.11: Composition and aggregation relationships



8.0 Deployment Diagram

Prepared by: Ngome Fernandez Manyaka

8.1 What is a Deployment Diagram?

A deployment diagram is a structural UML diagram that shows how software components are distributed across physical hardware. Where the class diagram shows what the code looks like, the deployment diagram shows where the code runs. It maps software artifacts, components, and their dependencies onto the physical or virtual execution environments that host them.

8.2 Architecture Overview

CrowdSenseNet follows a three-tier architecture as defined in the Task 3 SRS. The deployment diagram identifies five distinct execution environments across these three tiers.

Tier	Node	Role
1 (Client)	Android Device	Signal collection, local storage, user interface, background sync
1 (Client)	Android System Services	OS-level hardware APIs consumed by the app (TelephonyManager, FusedLocationProvider)
2 (Backend)	Backend Server	Data ingestion, validation, spatial storage, REST API, GeoJSON and CSV export
3 (Analytics)	ML Analytics Server	Coverage prediction, heatmap generation, model retraining (future phase)
External	Google Maps Tile Service	Third-party mapping tiles for the heatmap display layer

Table 8.1: Deployment nodes and their roles

8.3 Node Breakdown

8.3.1 Android Device

This node is the most complex in the diagram. It hosts the application code and all local data stores.

Components:

- Kotlin UI Application: the four-screen user interface (Home, Map, History, Settings) built with Material Design 3.

- **WorkManager Service:** the background sync agent that schedules uploads, enforces the WiFi-only constraint, and retries failed uploads every 30 minutes.
- **Adaptive Sampling Controller:** logic that monitors device speed and pauses collection when stationary for more than 30 seconds, resuming when speed exceeds 5 km/h. This is the primary mechanism for the battery efficiency requirement.
- **UUID Generator:** runs once on first launch to produce the anonymous device token attached to every reading.

Artifacts:

- **crowdsensenet.apk:** the installed Android application package.
- **Room Database (SQLite):** local offline data store with a 50MB storage cap.
- **uuid.prefs:** encrypted SharedPreferences file holding the anonymous UUID and the bearer authentication token.
- **settings.prefs:** user preference store for upload mode, sampling interval, and collection mode.

8.3.2 Android System Services

This node represents the Android OS hardware access layer. It is shown separately because CrowdSenseNet consumes these APIs at a defined interface boundary rather than owning them.

- **TelephonyManager:** exposes RSRP, RSRQ, SINR, RSSI, Cell ID, and network operator data from the cellular modem hardware.
- **FusedLocationProvider:** blends GPS, WiFi positioning, and cell tower triangulation to produce location fixes with an associated accuracy estimate. Readings with accuracy worse than 15 metres are discarded by the Kotlin app immediately after the fix is received.

8.3.3 Backend Server

The backend server runs Python FastAPI and PostgreSQL with the PostGIS spatial extension.

- **FastAPI REST API:** the sole entry point to the backend. Handles authentication, request routing, and the GeoJSON and CSV export endpoints used by network engineers.
- **Data Validator:** enforces the Cameroon bounding box filter, the duplicate GPS averaging rule (within 10 metres), and the authentication token check on every incoming batch.
- **PostGIS Query Engine:** executes spatial queries for heatmap grid generation, proximity clustering, and bounding box filtering directly within the PostgreSQL engine.

8.3.4 ML Analytics Server (Future Phase)

This node hosts the machine learning pipeline. It is included in the diagram because the data pipeline in Tiers 1 and 2 is being built specifically to feed it, even though it is a Won't Have (now) requirement from the Task 3 MoSCoW prioritisation.

- Coverage Prediction Model: classifies geographic grid cells as good, average, or coverage hole. Requires at least 50 readings per 1 km squared and must achieve at least 80% classification accuracy.
- Model Training Pipeline: periodically pulls fresh batches from PostgreSQL and retrains the model as more data accumulates.
- Heatmap Generator: converts prediction outputs into GeoJSON feature collections returned to the backend.

8.4 Inter-Node Communication Paths

From	To	Protocol	Description
Android Device	Backend Server	HTTPS / TLS 1.2+	WorkManager uploads validated reading batches as JSON. WiFi by default, mobile data opt-in. 30-minute retry on failure.
Backend Server	Android Device	HTTPS / TLS 1.2+	FastAPI returns the cached heatmap GeoJSON when the app Map screen requests it.
Backend Server	ML Analytics Server	Internal / batch	Scheduled bulk export of validated readings from PostgreSQL as a training data snapshot.
ML Analytics Server	Backend Server	Internal / API	Prediction GeoJSON returned after each prediction run. Backend caches it and serves it on the second path above.
Backend Server	Google Maps	HTTPS	Map tile requests for the base layer rendered under the coverage heatmap.

Table 8.2: Inter-node communication paths

8.5 Modelling Tool and Challenges

The deployment diagram was built using EdrawMax. An initial attempt in Modelio was abandoned after two technical blockers: artifacts could not be placed inside node boundaries using drag-and-drop, and the Component creation tool was absent from the default deployment diagram palette. EdrawMax resolved both issues and produced a clean, correctly structured diagram.

8.6 Deployment Diagram Illustration

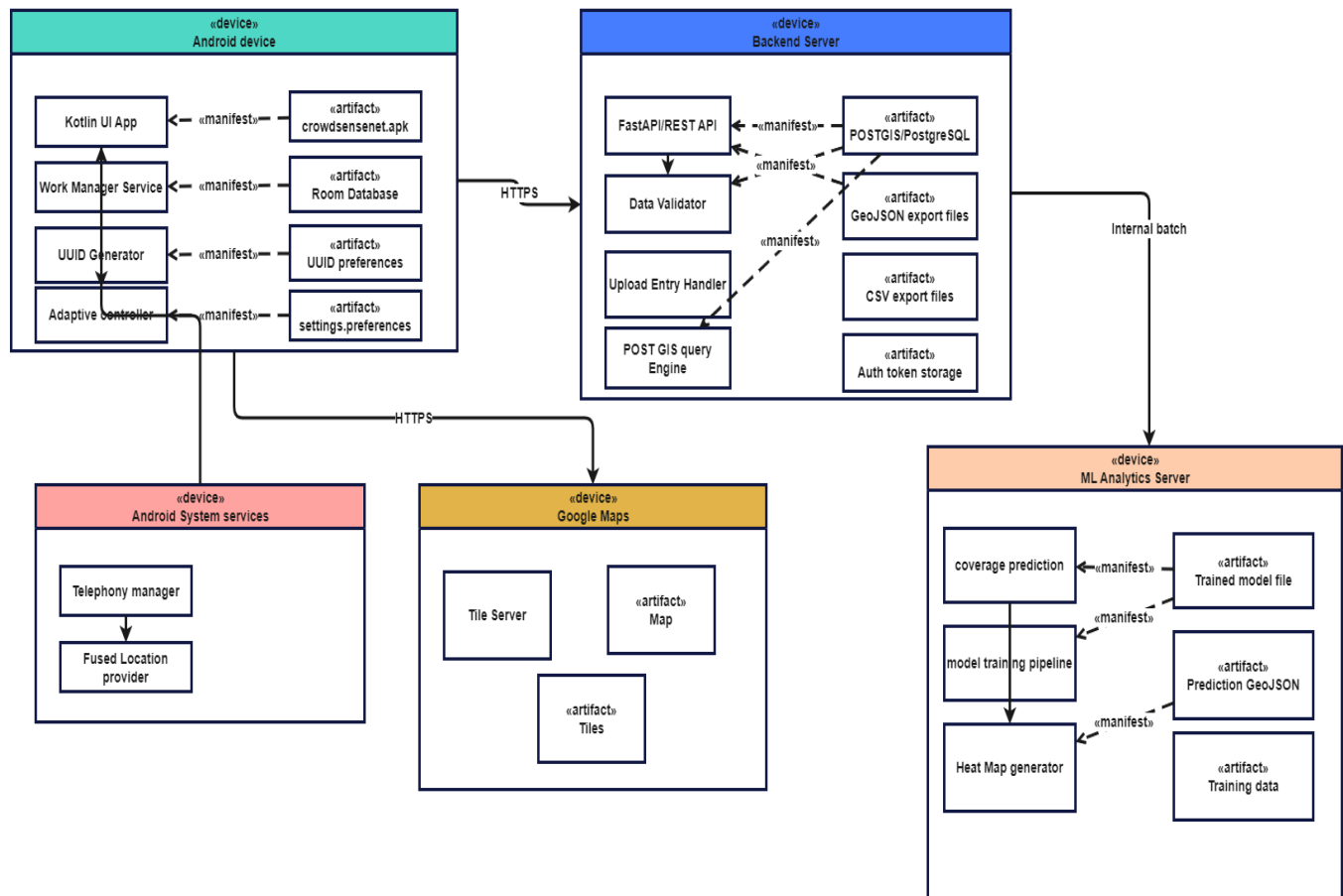


Figure 7: Deployment Diagram for CrowdSenseNet

9.0 Member Contributions

This section provides an honest account of each team member's contribution to Task 4. All five members participated in the shared tasks of reviewing the Task 3 SRS before starting, attending planning discussions, and cross-checking each other's diagrams for consistency. The individual contributions below reflect the primary ownership of each section.

Name	Student ID	Section Owned	Contribution Description	Estimated Participation
Tadjeu Ngnintedem Aubry-Junex	FE23A148	Context Diagram (Section 3)	Led the design and documentation of the context diagram. Identified all external actors, defined data flows, and wrote the full explanatory report. Validated the diagram against the Task 3 stakeholder list.	20%
Imele Azafa Jose de Kajos	FE23A068	Data Flow Diagram and Use Case Diagram (Sections 4 and 5)	Designed and documented both the Level 1 DFD and the Use Case Diagram. Defined all internal processes, data stores, actors, use cases, and relationships. These two diagrams required the most structural analysis work.	25%
Bessong Takang Tabot Willy	FE23A025	Sequence Diagram (Section 6)	Produced the sequence diagram for the core data collection scenario. Identified all lifelines, mapped the message flow across all four phases, documented design decisions with evidence from Task 2 data, and justified the combined fragments.	15%
Ayuk Amelia Agbor Orock	FE23A020	Class Diagram (Section 7)	Produced the complete class diagram covering all 29 classes, 2 interfaces, 3 enumerations, 7 packages,	25%

Name	Student ID	Section Owned	Contribution Description	Estimated Participation
			and 32 relationships following the MVVM pattern. Documented all attributes, operations, and relationship cardinalities.	
Ngome Fernandez Manyaka	FE23A111	Deployment Diagram (Section 8)	Designed and documented the deployment diagram across all five nodes. Identified components, artifacts, and communication paths. Led the EdrawMax modelling and documented the tool challenges encountered with Modelio.	20%

Table 9.1: Member contribution summary

Note on participation percentages: the slight variation between members reflects the difference in scope between sections. The Data Flow and Use Case Diagram section (Section 4 and 5) and the Class Diagram section (Section 7) involved the most analysis and documentation work, which accounts for their higher weighting. All members reviewed and approved each other's sections before submission.

10.0 Conclusion

This report has produced a complete system modelling and design for CrowdSenseNet, covering all six UML diagram types required by Task 4. Each diagram addresses a different layer of the system's design, and together they provide a full architectural blueprint ready for the implementation phase.

The context diagram established the system boundary and confirmed that all four stakeholder groups (users, network engineers, regulators, and academic supervisors) are properly accounted for. The data flow diagram opened the system boundary and mapped the seven internal processes through which raw signal readings become processed coverage maps. The use case diagram captured twelve functional interactions between the three primary actors and the system, including include and extend relationships that show the dependencies between features.

The sequence diagram modelled the runtime behaviour of the core data collection scenario across four phases, with explicit design decisions grounded in Task 2 evidence: the 10-second adaptive sampling interval, the 15-metre GPS accuracy filter, the on-device anonymisation step, and the WiFi-only upload default. The class diagram described the complete static structure of the Android application following the MVVM pattern, covering 29 classes, 2 interfaces, 3 enumerations, and 32 relationships across 7 packages. The deployment diagram mapped all software components onto five physical or virtual nodes across the three-tier architecture.

Every design decision in this report is traceable to a specific requirement in the Task 3 SRS and, behind that, to real stakeholder evidence from Task 2. The team is confident that this design correctly represents the system that needs to be built and provides a solid, evidence-based foundation for the implementation phase that follows.